

Τελικές Σημειώσεις DevOps Project

End-to-End Deployment FullStack Blogging Application με AWS EKS, Jenkins, SonarQube, Nexus, Trivy, ALB Ingress και HTTPS

1. Στόχος και τελική αρχιτεκτονική

Σκοπός του project είναι η αυτοματοποιημένη CI/CD διαδικασία για μία Spring Boot εφαρμογή, με build/test/security scan, publish artifact στο Nexus, Docker image push στο DockerHub και deployment σε AWS EKS.

```
Developer/GitHub
↓
Jenkins Pipeline
↓
Maven compile/test/package → SonarQube analysis → Trivy scan → Nexus deploy
↓
Docker build/push στο DockerHub
↓
EKS Deployment → Service ClusterIP → Ingress → AWS ALB
↓
HTTPS μέσω ACM certificate και domain app.nickzerze.com
```

2. Αρχικά EC2 instances και Security Groups

Δημιουργούνται χειροκίνητα 4 EC2 instances για τα DevOps tools. Τα EKS worker nodes δημιουργούνται από το Terraform/EKS Node Group, όχι χειροκίνητα.

EC2	Κύριες πόρτες inbound	Χρήση
Jenkins	22, 8080, 9100	SSH, Jenkins UI/webhooks, Node Exporter metrics
SonarQube	22, 9000, 9100	SSH, SonarQube UI και Jenkins analysis
Nexus	22, 8081, 9100	SSH, Nexus UI και Maven deploy από Jenkins
Monitor	22, 3000, 9090, 9115	SSH, Grafana, Prometheus, Blackbox Exporter

- Συνιστάται το SSH port 22 να είναι ανοιχτό μόνο από το δικό σου public IP.
- Για επικοινωνία μεταξύ EC2 instances μέσα στο ίδιο VPC, προτιμάται Security Group ως source αντί για 0.0.0.0/0.
- Τα passwords, tokens και private keys δεν μπαίνουν ποτέ σε Git. Χρησιμοποιούνται placeholders ή Jenkins credentials.

3. AWS CLI configuration

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
aws configure
aws sts get-caller-identity
```

Στο aws configure ορίζουμε Access Key, Secret Key, default region eu-central-1 και output format json.

4. EKS Cluster με Terraform

Μέσα στον φάκελο terraform υπάρχουν τα main.tf, variables.tf και output.tf. Το variables.tf περιέχει το ssh_key_name που πρέπει να υπάρχει ήδη στο AWS.

```
cd terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

- Το Terraform δημιουργεί VPC, subnets, route table, internet gateway, IAM roles, EKS cluster και managed node group.
- Για ALB Ingress χρειάζονται tags στα public subnets ώστε ο AWS Load Balancer Controller να τα ανακαλύπτει.

```
tags = {
  Name = "eks-full-stack-subnet-${count.index}"
  "kubernetes.io/cluster/eks-full-stack-cluster" = "shared"
  "kubernetes.io/role/elb" = "1"
}
```

5. Σύνδεση kubectl στο EKS

```
aws eks --region eu-central-1 update-kubeconfig --name eks-full-stack-cluster
kubectl get nodes
```

6. RBAC για Jenkins και DockerHub secret

Δημιουργούμε namespace webapps, ServiceAccount jenkins, Role, RoleBinding και Secret για token. Το token μπαίνει στο Jenkins ως Secret Text credential με ID π.χ. k8-cred.

```
kubectl create ns webapps
kubectl apply -f rbac/svc.yaml
kubectl apply -f rbac/role.yaml
kubectl apply -f rbac/bind.yaml
kubectl apply -f rbac/sec.yaml -n webapps
kubectl describe secret mysecretname -n webapps
```

Αν το DockerHub repository είναι private, δημιουργούμε imagePullSecret ώστε τα Pods να μπορούν να κάνουν pull το image.

```
kubectl create secret docker-registry regcred --docker-server=https://index.docker.io/v1/ --docker-username='malware4' --docker-password='DOCKERHUB_TOKEN_OR_PASSWORD' --docker-email='nickzerze@gmail.com' --namespace=webapps
kubectl get secret regcred --namespace=webapps --output=yaml
```

7. DevOps tools setup

7.1 SonarQube

```
sudo apt update
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
docker run -d --name Sonar -p 9000:9000 sonarqube:lts-community
```

- Access: http://<sonarqube-public-ip>:9000
- Default login: admin / admin και μετά αλλαγή password.
- Δημιουργούμε token στο SonarQube και το βάζουμε στο Jenkins ως Secret Text credential sonar-token.
- Στο SonarQube webhook βάζουμε: http://<jenkins-ip>:8080/sonarqube-webhook/

7.2 Nexus

```
docker run -d --name Nexus -p 8081:8081 sonatype/nexus3
docker ps
docker exec -it Nexus sh
cat /nexus-data/admin.password
```

- Access: http://<nexus-public-ip>:8081

- Repositories: maven-releases και maven-snapshots.
- Το pom.xml πρέπει να δείχνει στο δικό σου Nexus URL, όχι στο IP του οδηγού.

```
<distributionManagement>
  <repository>
    <id>maven-releases</id>
    <url>http://<NEXUS_IP>:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>maven-snapshots</id>
    <url>http://<NEXUS_IP>:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

8. Jenkins Pipeline και credentials

Στο Jenkins εγκαθίστανται plugins για Docker, Docker Pipeline, Kubernetes, Kubernetes CLI, Maven, SonarQube Scanner, Config File Provider, Pipeline Maven Integration και Email Extension.

Credential ID	Τύπος	Χρήση
git-cred	Username/password ή token	Checkout από GitHub
sonar-token	Secret text	SonarQube analysis και quality gate
docker-cred	Username/password	DockerHub login/push
k8-cred	Secret text	Kubernetes ServiceAccount token για deployment
mail-cred	Username/password	Email notification μέσω SMTP

8.1 Maven settings για Nexus

Στο Manage Jenkins → Managed Files δημιουργούμε Global Maven settings.xml με ID global-settings.

```
<settings>
  <servers>
    <server>
      <id>maven-releases</id>
      <username>admin</username>
      <password>NEXUS_PASSWORD</password>
    </server>
    <server>
      <id>maven-snapshots</id>
      <username>admin</username>
      <password>NEXUS_PASSWORD</password>
    </server>
  </servers>
</settings>
```

8.2 Docker client issue στο Jenkins

- Αν εμφανιστεί error “client version 1.29 is too old”, αφαιρούμε το toolName: docker από το withDockerRegistry.
- Εναλλακτικά κάνουμε manual docker login με withCredentials και --password-stdin.

```
withDockerRegistry(credentialsId: 'docker-cred') {
  sh "docker build -t malware4/bloggingapp:latest ."
}

withDockerRegistry(credentialsId: 'docker-cred') {
  sh "docker push malware4/bloggingapp:latest"
}
```

8.3 Trivy cache fix

Το /tmp ήταν tmpfs με μικρό χώρο. Γι' αυτό βάζουμε TMPDIR μέσα στο Jenkins workspace και χρησιμοποιούμε --cache-dir.

```
sh """
mkdir -p .trivy-cache trivy-tmp
```

```
TMPDIR=$PWD/trivy-tmp trivy fs --cache-dir .trivy-cache --scanners vuln --format table -o trivy-fs-report.html .
"""

sh """
mkdir -p .trivy-cache trivy-tmp
TMPDIR=$PWD/trivy-tmp trivy image --cache-dir .trivy-cache --scanners vuln --format table -o trivy-image-report.html malware4/bloggapp:latest
"""
```

9. Kubernetes Deployment: από LoadBalancer σε Ingress/ALB

Το αρχικό YAML με Service type LoadBalancer δημιουργεί απευθείας AWS Load Balancer για το Service. Το πιο production-like setup είναι Service type ClusterIP + Ingress + AWS Load Balancer Controller, ώστε το ALB να είναι το public entry point και τα Services/Pods να παραμένουν εσωτερικά.

```
Internet
↓
AWS ALB
↓
Ingress rules
↓
Service type ClusterIP
↓
Pods
```

9.1 deployment-ingress.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bloggapp-deployment
  namespace: webapps
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bloggapp
  template:
    metadata:
      labels:
        app: bloggapp
    spec:
      imagePullSecrets:
        - name: regcred
      containers:
        - name: bloggapp
          image: malware4/bloggapp:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: bloggapp-service
  namespace: webapps
spec:
  selector:
    app: bloggapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bloggapp-ingress
  namespace: webapps
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```

    alb.ingress.kubernetes.io/target-type: ip
spec:
  ingressClassName: alb
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: bloggingapp-service
            port:
              number: 80

```

10. AWS Load Balancer Controller

Ο controller παρακολουθεί Ingress resources και δημιουργεί/ενημερώνει AWS ALB. Χρειάζεται IAM permissions, OIDC provider, service account και Helm install.

```

aws eks --region eu-central-1 update-kubeconfig --name eks-full-stack-cluster

curl --silent --location "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz" |
tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

eksctl utils associate-iam-oidc-provider --region eu-central-1 --cluster eks-full-stack-cluster --approve

curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.14.1/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

eksctl create iamserviceaccount --cluster=eks-full-stack-cluster --namespace=kube-system --name=aws-load-balancer-controller --attach-policy-arn=arn:aws:iam::$ACCOUNT_ID:policy/AWSLoadBalancerControllerIAMPolicy --override-existing-serviceaccounts --region eu-central-1 --approve
helm repo add eks https://aws.github.io/eks-charts
helm repo update eks

VPC_ID=$(aws eks describe-cluster --region eu-central-1 --name eks-full-stack-cluster --query "cluster.resourcesVpcConfig.vpcId" --output text)

helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system --set clusterName=eks-full-stack-cluster --set region=eu-central-1 --set vpcId=$VPC_ID --set serviceAccount.create=false --set serviceAccount.name=aws-load-balancer-controller

kubectl get deployment -n kube-system aws-load-balancer-controller
kubectl logs -n kube-system deployment/aws-load-balancer-controller

```

- Αν εμφανιστεί “failed to get VPC ID from instance metadata”, κάνουμε helm upgrade δίνοντας explicit region και vpcId.
- Το deprecated annotation kubernetes.io/ingress.class αντικαθίσταται από spec.ingressClassName: alb.

11. HTTPS με Namecheap domain και AWS ACM

Για trusted HTTPS χρειάζεται domain. Στο project χρησιμοποιήθηκε το nickzerze.com από Namecheap και subdomain app.nickzerze.com.

1. Στο AWS Certificate Manager, στο region eu-central-1, ζητάμε Public Certificate για app.nickzerze.com.
2. Επιλέγουμε DNS validation.
3. Παίρνουμε το CNAME name/value από το ACM.
4. Στο Namecheap → Domain List → nickzerze.com → Manage → Advanced DNS προσθέτουμε CNAME record για validation.
5. Περιμένουμε μέχρι το ACM certificate να γίνει Issued.

6. Αντιγράφουμε το certificate ARN και το βάζουμε στο Ingress.

```
# Παράδειγμα Namecheap ACM validation record
Type: CNAME Record
Host: _xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.app
Value: _yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy.zzzzzzzzz.acm-validations.aws
TTL: Automatic
```

Στο Namecheap το Host δεν πρέπει να είναι ολόκληρο το FQDN. Για app.nickzerze.com βάζουμε μόνο το κομμάτι πριν το .nickzerze.com, π.χ. _xxxx.app.

12. Ingress με HTTPS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bloggingapp-ingress
  namespace: webapps
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-central-1:ACCOUNT_ID:certificate/CERTIFICATE_ID
    alb.ingress.kubernetes.io/ssl-redirect: '443'
spec:
  ingressClassName: alb
  rules:
    - host: app.nickzerze.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: bloggingapp-service
                port:
                  number: 80
kubectl apply -f deployment-ingress-https.yaml
kubectl get ingress -n webapps
kubectl describe ingress bloggingapp-ingress -n webapps
```

12.1 DNS record για την εφαρμογή

Αφού δημιουργηθεί ο ALB, παίρνουμε το ADDRESS από kubectl get ingress και στο Namecheap προσθέτουμε CNAME:

```
Type: CNAME Record
Host: app
Value: k8s-webapps-bloggingapp-xxxx.eu-central-1.elb.amazonaws.com
TTL: Automatic
```

Μετά τους DNS χρόνους propagation, η εφαρμογή ανοίγει από:

```
https://app.nickzerze.com
```

13. Έλεγχοι λειτουργίας

```
kubectl get pods -n webapps -o wide
kubectl get svc -n webapps
kubectl get ingress -n webapps
kubectl describe ingress bloggingapp-ingress -n webapps
```

```
curl -I http://app.nickzerze.com
curl -I https://app.nickzerze.com
```

```
dig app.nickzerze.com CNAME @8.8.8.8 +short
dig app.nickzerze.com CNAME @1.1.1.1 +short
```

- HTTP πρέπει να επιστρέφει 301 redirect προς HTTPS.
- HTTPS μπορεί να επιστρέφει 302 προς /login, που σημαίνει ότι η εφαρμογή απαντά κανονικά.

- Αν το DNS κάποιες φορές δεν κάνει resolve, περιμένουμε propagation ή καθαρίζουμε DNS cache σε Windows/browser.

14. Cleanup και έλεγχος χρεώσεων

Πριν το terraform destroy διαγράφουμε πρώτα τα Kubernetes resources ώστε να διαγραφούν ALB/ENIs που δημιουργήθηκαν από τον controller.

```
kubect1 delete -f deployment-ingress-https.yaml
kubect1 delete ns webapps
terraform destroy
```

Resource	Μπορεί να χρεώνει;	Έλεγχος
ACM public certificate	Όχι συνήθως	Μπορεί να μείνει ή να διαγραφεί για καθαριότητα
ALB / ELB	Ναι	aws elbv2 describe-load-balancers --region eu-central-1
EKS cluster	Ναι	aws eks list-clusters --region eu-central-1
EC2 instances	Ναι	aws ec2 describe-instances --region eu-central-1
EBS volumes	Ναι	aws ec2 describe-volumes --region eu-central-1
Elastic IPs	Ναι αν unattached	aws ec2 describe-addresses --region eu-central-1
NAT Gateways	Ναι	aws ec2 describe-nat-gateways --region eu-central-1
Route 53 Hosted Zone	Ναι	aws route53 list-hosted-zones

```
aws eks list-clusters --region eu-central-1
aws elbv2 describe-load-balancers --region eu-central-1 --output table
aws ec2 describe-instances --region eu-central-1 --output table
aws ec2 describe-volumes --region eu-central-1 --output table
aws ec2 describe-addresses --region eu-central-1 --output table
aws ec2 describe-nat-gateways --region eu-central-1 --output table
aws route53 list-hosted-zones
```

15. Συχνά προβλήματα και λύσεις

Πρόβλημα	Αιτία	Λύση
Terraform undeclared resource	Μη 一致 resource labels με hyphen/underscore	Χρησιμοποίησε παντού ίδιο label, κατά προτίμηση underscores
Nexus insufficient memory	Μικρό EC2 instance/RAM	Χρήση t3.medium ή περιορισμός JVM heap
Nexus 401 Unauthorized	Λάθος Maven settings ή credentials	Τα ids maven-releases/maven-snapshots πρέπει να ταιριάζουν με pom.xml
Docker client 1.29 too old	Jenkins Docker tool κατεβάζει παλιό client	Αφαίρεση toolName: docker από withDockerRegistry
Trivy no space left on device	/tmp mounted ως μικρό tmpfs	Χρήση TMPDIR=\$PWD/trivy-tmp και --cache-dir
Ingress no ADDRESS	ALB controller/subnet tags/IAM issue	Έλεγχος controller logs, IAM SA, OIDC, subnet tags
DNS_PROBE_POSSIBLE	DNS propagation/cache	dig/nslookup, flush DNS, έλεγχος Namecheap records